

Unit 1: Python Programming Fundamentals

1.1 Python Introduction

Python is a powerful, high-level, and general-purpose programming language. It was created by **Guido van Rossum** and first released in **1991**. Its design philosophy emphasizes **code readability**, which makes it an excellent choice for beginners and professionals alike.

Features of Python:

- **Simple and Readable Syntax:** Python code resembles English, making it easier to read and write.
- **Interpreted Language:** Python executes code line by line, which helps in debugging and testing.
- **Dynamically Typed:** You don't need to declare the data type of a variable explicitly.
- **Object-Oriented:** Supports object-oriented programming principles such as classes and inheritance.
- **Cross-platform:** Python runs on various platforms including Windows, Linux, and macOS.
- **Large Standard Library:** Comes with built-in modules for file handling, regular expressions, math, web services, and more.
- **Extensive Community Support:** There is a vast number of tutorials, forums, and third-party libraries like NumPy, Pandas, TensorFlow, Django, Flask, etc.

1.2 Data Types and Type Conversion

In Python, **data types** are classifications that determine what kind of value a variable can hold. Python is **dynamically typed**, which means you don't have to declare the type of a variable explicitly—Python detects it automatically.

Common Built-in Data Types:

Data Type	Description	Example
int	Integer numbers	x = 10
float	Decimal numbers	pi = 3.14

str	String of characters	name = "Alice"
bool	Boolean values (True or False)	is_valid = True
list	Ordered collection (mutable)	numbers = [1, 2, 3]
tuple	Ordered collection (immutable)	t = (1, 2, 3)
dict	Key-value pairs	person = {'name': 'Bob'}
set	Unordered collection of unique elements	s = {1, 2, 3}

Type Conversion (Type Casting):

Type conversion refers to changing the data type of a value. Python provides built-in functions for type casting:

Function	Purpose	Example
int()	Converts to integer	int("10") → 10
float()	Converts to float	float("3.14") → 3.14
str()	Converts to string	str(123) → "123"
bool()	Converts to boolean	bool(0) → False
list(), tuple()	Converts iterable types	list("abc") → ['a','b','c']

1.3 Comments

Comments in Python are used to explain code, make it more readable, and help developers understand the logic behind specific code blocks. Python ignores comments during program execution—they are only for human readers.

1.4 Variables, Constants, Operators and Performing Calculations

1. Variables

A **variable** is a named storage in memory that holds a value. In Python, variables are created by simply assigning a value to a name.

Syntax:

```
variable_name = value
```

2. Constants

A **constant** is a variable whose value should not change throughout the program. Python does not have built-in constant types, but naming conventions are used to signal constants.

Example:

PI = 3.14159

MAX_USERS = 100

3. Operators

Operators are symbols that perform operations on variables and values.

► *Arithmetic Operators:*

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division (float)	a / b
//	Floor division	a // b
%	Modulus (remainder)	a % b
**	Exponentiation	a ** b

► *Assignment Operators:*

Operator	Example	Meaning
=	x = 5	Assign 5 to x
+=	x += 3	x = x + 3
-=	x -= 2	x = x - 2

► *Comparison Operators:*

Used to compare values; result is True or False.

Operator	Description	Example
==	Equal	a == b
!=	Not equal	a != b

>	Greater than	a > b
<	Less than	a < b
>=	Greater or equal	a >= b
<=	Less or equal	a <= b

4. Performing Calculations

Examples:

```
a = 10 b = 3
```

```
sum_result = a + b # 13
```

```
diff = a - b # 7
```

```
product = a * b # 30
```

```
quotient = a / b # 3.333...
```

```
modulus = a % b # 1
```

```
power = a ** b # 1000
```

1.5 Reading Input from Keyboard

Reading input from the user is essential for interactive Python programs. Python provides the built-in `input()` function to accept data entered through the keyboard.

Using `input()` Function

Syntax:

```
variable = input("Enter a value: ")
```

- The string inside the `input()` function is displayed as a prompt.
- The input is always returned as a **string**, even if the user types a number.

Converting Input to Other Data Types

To use numerical input for calculations, convert it using `int()` or `float()`:

```
num1 = int(input("Enter first number: "))  
num2 = int(input("Enter second number: "))  
sum = num1 + num2  
print("Sum is:", sum)
```

```
price = float(input("Enter product price: "))  
quantity = int(input("Enter quantity: "))  
total = price * quantity  
print("Total cost:", total)
```

What is an f-string in Python?

f-strings (formatted string literals) are a way to embed expressions (like variables or calculations) directly inside string literals using curly braces `{ }`. They were introduced in **Python 3.6** for easier and more readable string formatting.

1.6 Print Function and Displaying Formatted Output with F-strings

The `print()` Function

The `print()` function is used to display output on the screen. It can print strings, numbers, variables, and expressions.

Syntax:

```
print(object(s), sep=' ', end='\n')
```

- `sep` (separator): defines how values are separated (default is a space).
- `end`: defines what to print at the end (default is a newline `\n`).

Formatted Strings (f-strings)

Introduced in **Python 3.6**, **f-strings** offer a concise way to include variables and expressions inside strings.

Syntax:

```
f"Your message with {variable}"
```

Formatting with Precision and Width:

Floating point formatting:

```
pi = 3.14159265  
print(f"Value of pi is {pi:.2f}") # Output: 3.14
```

Comparison: + vs. f-strings

Using + for string concatenation:

```
name = "Alice"  
print("Hello, " + name + "!")
```

Advantages of f-strings:

- Easier to read and write
- More efficient
- Supports expressions and formatting